目录

2 进程与线程管理 2
2.1 概述 2
2.2 核心设计思想 3
2.3 关键数据结构 4
2.4 进程与线程创建 5
2.5 程序执行 6
2.6 退出与资源回收 7
2.7 进程组与会话 8
2.8 权限与进程控制 10
2.9 本章小结 12
3 调度与同步机制 13
3.1 概述 13
3.2 任务调度机制 13
3.3 阻塞与唤醒机制 14
3.4 定时与超时机制 15
3.5 Futex 同步机制 16
3.6 子进程状态与等待 17
3.7 本章小结 18

2

2.1

2.1.1 StarryOS

StarryOS `Thread` `ProcessData`

StarryOS `fork` `clone` `clone3` `execve` `exit` `waitpid` pidfd moss Linux

2.1.2 base os

“ ” StarryOS

`clone3` pidfd `execve` `execveat` `MS_NOEXEC`

`waitpid` “ ” “ ” `waitid` `siginfo_t` UID/GID capability `kill` ptrace pidfd `waitid` `/proc`

2.1.3 moss

moss StarryOS

	StarryOS	moss
	`Thread` `ProcessData`	/ namespace
	`clone3` `execve`	clone ABI `execveat`
	`waitpid`	`waitid` pidfd
	pidfd	ptrace

2-1

2.2

StarryOS moss

2.2.1

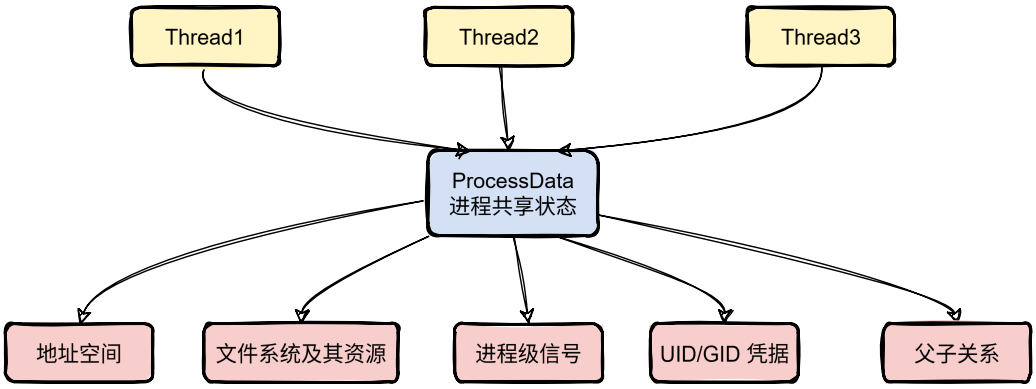
moss StarryOS

`Thread` `ProcessData`

`ProcessData` clone

`Thread`

`ProcessData` clone exec exit



2-1 Thread ProcessData

2.2.2

moss

- `ProcessData`
- parent children wait
- PID PGID SID

“ ” “ ”

“ ” moss

2.2.3

Linux UID real effective saved real ID

effective ID saved ID GID

moss UID/GID `ProcessData` `setreuid` `setresuid`

ptrace

2.2.4



exec PID

exit wait moss

2.3

2.3.1 Thread ProcessData

`Thread` `ProcessData`

```
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}

pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    uid: AtomicU32,
    euid: AtomicU32,
    suid: AtomicU32,
}
```

2-1

`Arc<ProcessData>`

`Thread` moss

2.3.2

StarryOS moss

`ProcessWaitStatus`

```
pub enum ProcessWaitStatus {
    // 被信号暂停，但进程仍然存在
    Stopped(Signo),
    // 从停止状态恢复
    Continued(Signo),
}
```

2-2

Process ProcessData

“ ” “ ” waitpid waitid

2.3.3

TID PID PGID SID moss

TASK_TABLE	TID	
PROCESS_TABLE	PID	
PROCESS_GROUP_TABLE	PGID	
SESSION_TABLE	SID	

2-2

moss

PGID SID

2.4

2.4.1 fork clone

fork moss x86_64 sys_fork sys_clone

SIGCHLD fork ProcessData

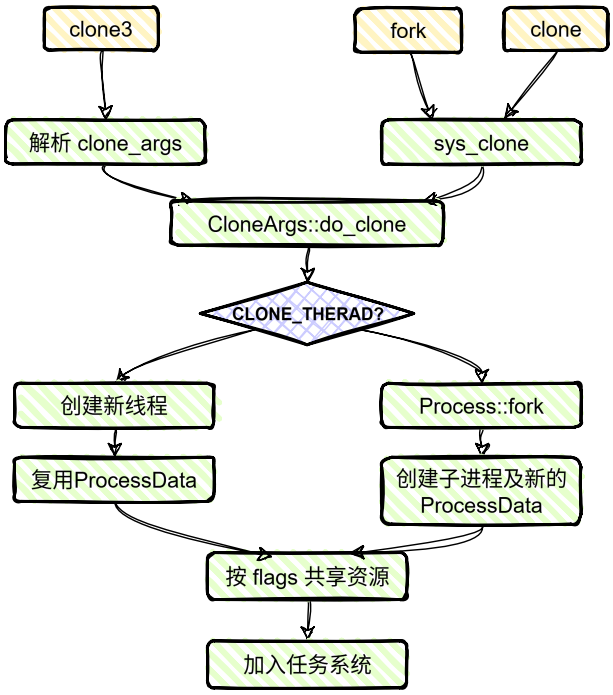
clone clone3

CloneArgs CloneArgs::do_clone

CLONE_THREAD ProcessData Process::fork

ProcessData

CLONE_VM CLONE_FILES CLONE_FS CLONE_SIGHAND



2-2 fork clone clone3

CLONE_THREAD	ProcessData
CLONE_VM	
CLONE_FILES	
CLONE_FS	
CLONE_SIGHAND	

2-3 clone

CLONE_THREAD

2.4.2

moss clone

```
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

2-3 clone

clone3 moss

2.4.3

pidfd

pidfd

moss

“ PID ”

CLONE_VFORK namespace

2.5

2.5.1 execve

StarryOS execve ELF

CLOEXEC exec

PID

MS_NOEXEC

2.5.2 execveat

execveat dirfd

AT_EMPTY_PATH

AT_SYMLINK_NOFOLLOW

execve

execveat AT_EMPTY_PATH

Linux execve

moss execve execveat execve_common

moss

```
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

2-4

2.5.3 exec

Linux exec

moss exec

2.6

2.6.1

robust futex clear_child_tid

ProcessData

moss

ptrace `wait`

2.6.2 exit_group

2.6.1 `exit_group`

moss

2.6.1

`exit` `exit_group`

2.6.3

StarryOS `children` init

→ init →

PID

init `wait`

2.7

2.7.1 PID PGID SID

PID PGID

SID PID PGID PID SID

Shell

moss StarryOS `Process` PID `ProcessGroup`

`ProcessGroup` PGID `Session` `Session` SID

```rust
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

2-5

```rust
// 先按 PID 找到进程对象
let proc = get_process_data(pid)?.proc;
let pid  = proc.pid();
// 沿所属关系取得进程组和会话标识
let pgid = proc.group().pgid();
let sid  = proc.group().session().sid();
```

2-6 PID PGID SID

ProcessGroup::processes moss PROCESS_TABLE PROCESS_GROUP_TABLE SESSION_TABLE PID PGID SID

2.7.2

StarryOS setpgid getpgid PGID

PGID

moss PGID PGID PID

getpgid

PGID

PGID

2.7.3

`setsid` moss `Session` `ProcessGroup`

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
    Ok(session.sid() as _)
}
```

2-7

PID PGID SID

2.8

2.8.1 UID/GID

Linux UID real effective saved

real UID effective UID

saved UID GID

set-user-ID

effective UID saved UID

UID UID

StarryOS moss `ProcessData`

UID/GID `setresuid` `setresgid`

UID

```
pub struct ProcessData {
    uid: AtomicU32,  // real UID: 进程的实际身份
    euid: AtomicU32, // effective UID: 当前权限判断使用
    suid: AtomicU32, // saved UID: 保留可恢复的身份
}

let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}

if ruid != UID_GID_NO_CHANGE {
    proc_data.set_real_uid(ruid);
}
if euid != UID_GID_NO_CHANGE {
    proc_data.set_euid(euid);
}
if suid != UID_GID_NO_CHANGE {
    proc_data.set_suid(suid);
}
```

2-8 UID

effective UID 0

effective UID/GID `kill` ptrace attach

capability effective permitted inheritable bounding

2.8.2 ptrace

moss ptrace `TRACEME` `ATTACH` `DETACH` wait

attach

wait

`PTRACE_SYSCALL` `PTRACE_SINGLESTEP`

siginfo `EIO` GDB

2.8.3 namespace

moss PID UTS User namespace inode namespace init PID

clone unshare PID UID

namespace

2.9

moss StarryOS `Thread` `ProcessData` real effective saved ID clone exec exit wait

moss `clone3` ABI `execveat`

ptrace

vfork exec namespace ptrace

StarryOS

3

CPU

Futex

3.1

StarryOS `axtask` `axsched`
Future Waker

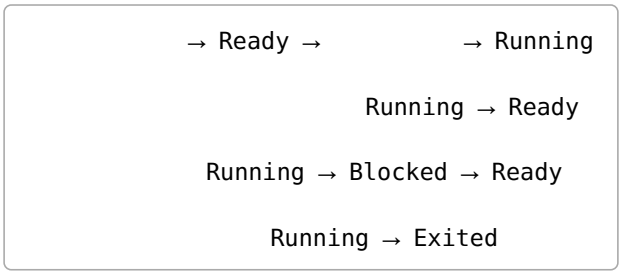
moss

`waitid`
CPU moss
Linux PollSet
Futex

3.2

3.2.1

- `Running` CPU
- `Ready`
- `Blocked`
- `Exited`



3-1

`Ready` `Running` CPU
`Ready` `Blocked`

CPU CPU
CPU

3.2.2

RR
5 CPU

`scheduler_timer_tick`



CPU idle

3.2.3

	`Running → Ready`	
`yield_now()`	`Running → Ready`	CPU
	`Running → Blocked`	

3-1

`switch_to`

`TaskContext` CPU

StarryOS

RR PID Linux

3.2.4 CPU

StarryOS CPU moss

`sched_getaffinity` CPU `sched_setaffinity` CPU

`set_current_affinity` CPU

`sched_yield` `sched_getscheduler`

RR moss

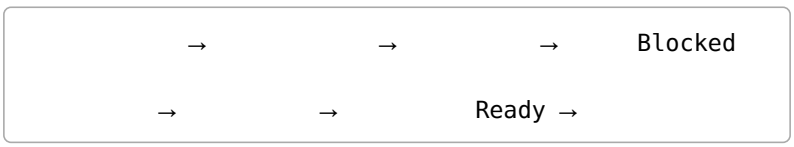
I/O CPU

3.3

3.3.1

`yield_now()`

`Blocked`



3-2

“ ”

3.3.2 Waker PollSet

Waker

Waker `wake()` Waker `Blocked` `Ready`

`PollSet` Waker eventfd timerfd pidfd

moss `has_waiters`

Waker

PollSet “ ”

Futex

3.3.3

`block_on` Future Future `Pending` `block_on`

`Blocked` Waker `Ready` CPU

Future

Waker `woke`

CPU

Waker

`interruptible` Future

`EINTR`

3.3.4

Waker

Waker moss PollSet

Futex 3.5

3.4

3.4.1

“ ”

`clock_nanosleep` Futex

3.4.2

moss `(deadline, seq)` `BTreeMap` `deadline` `seq`

alarm task

interval timer `TimerKey` Key

Key

`EVENT_NEW_TIMER` alarm task

3.4.3

`nanosleep` `clock_nanosleep` `TIMER_ABSTIME` moss

`EINTR` `rem`

LoongArch64

`yield_now()`

`Blocked` Futex

`timeout` Future

3.4.4

`TimeManager`

`times`

interval timer `ITIMER_REAL` `SIGALRM`

`ITIMER_VIRTUAL` `SIGVTALRM` `ITIMER_PROF`

`SIGPROF` interval

3.5 Futex

3.5.1

Futex

`FutexKey` Futex

Futex

Futex `owner_dead` moss

`notified` `bitset` `PollSet`

3.5.2 WAIT WAKE

`FUTEX_WAIT`

`EAGAIN`

Future `FUTEX_WAKE`

`notified` PollSet

wake

Waker `notified`

moss Waker

3.5.3

`FUTEX_WAIT_BITSET` `FUTEX_WAKE_BITSET` bitset mask

Futex

`FUTEX_REQUEUE` Futex `FUTEX_CMP_REQUEUE` `EAGAIN` `notified` bitset PollSet Waker pthread

3.5.4

Futex moss `interruptible(timeout(...))` `ETIMEDOUT` `EINTR` `notified` `FUTEX_WAIT` `FUTEX_WAIT_BITSET` moss Linux `FUTEX_CLOCK_REALTIME`

LoongArch64 `futex_wait` pthread cancellation

3.5.5 robust list

`set_robust_list` robust list Futex moss `futex_offset` Futex `owner_dead` `EOWNERDEAD` `ROBUST_LIST_LIMIT` `clear_child_tid` Futex `pthread_join`

3.6

3.6.1

StarryOS `waitpid` moss `ProcessData` `SIGCONT`

3.6.2

wait Waker `child_exit_event` `wake()` 3.3

`interruptible` `EINTR`

3.6.3 wait

`waitpid` pid moss

`Pid` `Pgid` `Any`

`ECHILD`

`WNOHANG`	
`WUNTRACED`	
`WCONTINUED`	
`WNOWAIT`	

3-2

`wait4` `waitid` `siginfo_t`

`CLD_EXITED` `CLD_KILLED` `CLD_STOPPED` `CLD_CONTINUED`

3.6.4 waitid pidfd

moss `waitid` `P_ALL` `P_PID` `P_PGID` `P_PIDFD` PID

`P_PIDFD`

pidfd StarryOS moss `/proc` fdinfo PID

pidfd pidfd `EAGAIN`

pidfd

`WNOWAIT`

3.7

StarryOS RR moss

PollSet

`TimerKey`

moss Futex robust list

`waitid` pidfd